

ACM-mode

单行读入

`input()`, 配合 `split`, `map`, `split` 等函数。

多行读入

```
for line in sys.stdin:
```

Exercise 1: 接雨水 - 一维数组

```
from collections import Counter

class Solution(object):
    def findAnagrams(self, s, p):
        p_map = Counter(p)
        s_map = Counter()
        left, right = 0, 0
        res = []
        while right < len(s):
            s_map[s[right]] += 1
            while left < right and s_map > p_map: # 滑动窗口的字符覆盖子串p
                s_map[s[left]] -= 1
                left += 1
            if s_map == p_map:
                res.append(left)
            right += 1
        return res

if __name__ == "__main__":
    s = input()
    p = input()
    solution = Solution()
    res = solution.findAnagrams(s, p)
    print(res)
```

Exercise2: 找到字符串中的所有字母异位词 - 字符串

优化前:

```
class Solution(object):
    def findAnagrams(self, s, p):
        p_map = Counter(p)
        s_map = Counter()
        left, right = 0, 0
        res = []
        while right < len(s):
            s_map[s[right]] += 1
            while left < right and s_map > p_map: # 滑动窗口的字符覆盖子串p
                s_map[s[left]] -= 1
```

```

        left += 1
        if s_map == p_map:
            res.append(left)
        right += 1
    return res

if __name__ == "__main__":
    s = input()
    p = input()
    solution = Solution()
    res = solution.findAnagrams()
    print(res)

```

优化后:

```

from collections import Counter

class Solution(object):
    def findAnagrams(self, s, p):
        p_map = Counter(p)
        left, right = 0, 0
        res = []
        for right in range(len(s)):
            p_map[s[right]] -= 1
            while p_map[s[right]] < 0:
                p_map[s[left]] += 1
                left += 1
            if right - left + 1 == len(p):
                res.append(left)
        return res

if __name__ == "__main__":
    s = input()
    p = input()
    solution = Solution()
    res = solution.findAnagrams(s, p)
    print(res)

```

Exercise 3: 螺旋矩阵 - 二维数组

```

import sys
class Solution(object):
    def spiralOrder(self, matrix):
        m, n = len(matrix), len(matrix[0])
        res = []
        top, bottom, left, right = 0, m-1, 0, n-1
        i, j = 0, 0
        while top <= bottom and left <= right:
            # 在top行向右走, 从left到right
            for j in range(left, right+1):
                res.append(matrix[top][j])
                j += 1

```

```

        # 在right列向下走, 从top+1到bottom
        for i in range(top+1, bottom+1):
            res.append(matrix[i][right])
        # 如果此时缩成一行/一列要返回, 否则重复遍历
        if top >= bottom or left >= right:
            break
        # 在bottom行向左走, 从right-1到left
        for j in range(right-1, left-1, -1):
            res.append(matrix[bottom][j])
        # 在left列向上走, 从bottom-1到top+1
        for i in range(bottom-1, top, -1):
            res.append(matrix[i][left])
        left, right, top, bottom = left+1, right-1, top+1, bottom-1
    return res

if __name__ == "__main__":
    matrix = []
    for line in sys.stdin:
        matrix.append(list(map(int, line.split()))))
    solution = Solution()
    res = solution.spiralOrder(matrix)
    print(res)

```

Exercise 4: 排序链表 - 链表

```

import sys
class ListNode(object):
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution(object):
    def sortList(self, head):
        def mergeList(head1, head2):
            # 给定两个升序链表头head1和head2, 合并成一个升序链表
            dummy = ListNode()
            cur = dummy
            p1, p2 = head1, head2
            while p1 and p2:
                if p1.val < p2.val:
                    cur.next = p1
                    cur = cur.next
                    p1 = p1.next
                else:
                    cur.next = p2
                    cur = cur.next
                    p2 = p2.next
            cur.next = p1 if p1 else p2
            return dummy.next

        # 主程序: 递归实现
        if not head or not head.next: # 单节点链表或空链表

```

```

        return head
    slow, fast = head, head
    # 快慢指针找中点, slow会停在中点(奇数)或后半段第一个点(偶数)
    while fast and fast.next:
        pre = slow
        slow = slow.next
        fast = fast.next.next
    # 断开前半段和后半段
    pre.next = None
    head = self.sortList(head)
    slow = self.sortList(slow)
    return mergeList(head, slow)

if __name__ == "__main__":
    dummy = ListNode()
    cur = dummy
    for line in sys.stdin:
        cur.next = ListNode(val=int(line))
        cur = cur.next
    solution = Solution()
    dummy.next = solution.sortList(dummy.next)
    cur = dummy.next
    while cur:
        print(cur.val)
        cur = cur.next

```

Exercise 5: LRU 缓存 - 自定义类

```

from collections import defaultdict

class ListNode(object): # 双向链表的节点
    def __init__(self, key=0, value=0, next=None, prev=None):
        self.key = key
        self.value = value
        self.next = next
        self.prev = prev

class LRUCache(object):
    def __init__(self, capacity):
        self.c = capacity
        self.size = 0 # 当前链表长度
        self.head = ListNode() # 虚拟头节点
        self.rear = ListNode() # 虚拟尾节点
        self.head.next, self.rear.prev = self.rear, self.head # 初始化
        self.node_map = defaultdict(ListNode) # 哈希表

    def moveHead(self, node):
        # 把node移动到链表头
        node.next, node.prev = self.head.next, self.head
        self.head.next, node.next.prev = node, node

    def removeNode(self, node):

```

```

# 删除node节点
node.prev.next, node.next.prev = node.next, node.prev

def put(self, key, value):
    if key in self.node_map:
        node = self.node_map[key]
        node.value = value
        # 删除原node, 把node放到链表头
        self.removeNode(node)
        self.moveHead(node)
        return
    node = ListNode(key=key, value=value)
    self.node_map[key] = node
    # 新节点放到链表头
    self.moveHead(node)
    if self.c == self.size:
        # 链表满了, 删掉最后一个节点
        last_node = self.rear.prev
        del self.node_map[last_node.key]
        self.removeNode(last_node)
    else:
        self.size += 1

def get(self, key):
    # 如果key不存在
    if key not in self.node_map:
        return -1
    node = self.node_map[key]
    # 删除原node, 把node放到链表头
    self.removeNode(node)
    self.moveHead(node)
    return node.value

if __name__ == "__main__":
    lru = LRUCache(3)
    lru.put(1, 100)
    lru.put(2, 200)
    lru.put(3, 300)
    print(lru.get(1)) # 应输出100
    lru.put(4, 400)
    print(lru.get(2)) # 应输出-1
    print(lru.node_map.keys()) # 应输出dict_keys([1, 3, 4]), 说明2被删除了

```

Exercise 6: 二叉树的最近公共祖先 - 树

```

import sys
from collections import deque

class TreeNode(object):
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left

```

```
self.right = right
```

```
class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        if not root or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        # 同时找到了返回交汇点, 否则返回不为None的那个
        # 如果都是None定义返回right, 没影响, 因为说明p和q不在这棵子树上
        if left and right:
            return root
        return left if left else right

def buildTree(nums, p_num, q_num):
    if not nums:
        return None
    # 根节点入队
    root = TreeNode(int(nums[0]))
    if root.val == p_num:
        p = root
    if root.val == q_num:
        q = root
    queue = deque([root])
    i, n = 1, len(nums)
    while i < n:
        node = queue.popleft()
        # 左孩子
        if nums[i] != "None":
            node.left = TreeNode(int(nums[i]))
            queue.append(node.left)
            if node.left.val == p_num:
                p = node.left
            if node.left.val == q_num:
                q = node.left
        i += 1
        # 右孩子
        if i < n and nums[i] != "None":
            node.right = TreeNode(int(nums[i]))
            queue.append(node.right)
            if node.right.val == p_num:
                p = node.right
            if node.right.val == q_num:
                q = node.right
        i += 1
    return root, p, q

if __name__ == "__main__":
    solution = Solution()
    # 第一行按满二叉树顺序一行写入, 第二行以空格隔开p和q
    nums = list(input().split()) # nums暂时还是字符串
    p_num, q_num = map(int, input().split()) # p_num和q_num是数字
    root, p, q = buildTree(nums, p_num, q_num) # p和q是节点
```

```
print(solution.lowestCommonAncestor(root, p, q).val)
```

Exercise 7: 数组中的第K个最大元素 - 自定义方法

```
class Solution(object):
    def findKthLargest(self, nums, k):
        def max_heapify(i, heap, heap_size):
            # 对以i为根节点的堆进行堆调整
            left, right = 2*i+1, 2*i+2
            largest = i
            if left < heap_size and heap[left] > heap[largest]:
                largest = left
            if right < heap_size and heap[right] > heap[largest]:
                largest = right
            if largest != i:
                max_heapify(largest, heap, heap_size)

        def build_max_heap(heap, heap_size):
            for i in range(heap_size-1, -1, -1):
                max_heapify(i, heap, heap_size)

        # 主程序: 排k次
        heap = build_max_heap(nums, len(nums))
        for i in range(k-1): # k-1次后, 堆顶就是第k大
            heap[0], heap[-(i+1)] = heap[-(i+1)], heap[0]
            max_heapify(0, heap, heap_size-(i+1))
        return heap[0]

if __name__ == "__main__":
    nums = list(map(int, input().split()))
    solution = Solution()
    print(solution.findKthLargest(nums))
```

Others

排序数组 (No.912)

1. 快速排序

```
import random

class Solution:
    def sortArray(self, nums):
        def partition(nums, left, right):
            # nums[left:right+1]之间排序
            pivot_index = random.randint(left, right)
            pivot = nums[pivot_index]
            # 把pivot换到第一个数
            nums[pivot_index], nums[left] = nums[left], nums[pivot_index]
            i, j = left + 1, right
            while True:
```

```

        while i <= j and nums[i] < pivot:
            i += 1
        while i <= j and nums[j] > pivot:
            j -= 1
        if i >= j:
            break
        nums[i], nums[j] = nums[j], nums[i]
        i += 1
        j -= 1
    nums[j], nums[left] = nums[left], nums[j]
    return j

def quickSort(nums, left, right):
    # 对nums[left:right+1]递归地快速排序
    if left >= right:
        return nums
    i = partition(nums, left, right)
    quickSort(nums, left, i - 1)
    quickSort(nums, i + 1, right)

quickSort(nums, 0, len(nums)-1)
return nums

if __name__ == "__main__":
    nums = list(map(int, input().split()))
    solution = Solution()
    print(solution.sortArray(nums))

```

2. 归并排序

```

class Solution(object):
    def mergeArray(self, left, right):
        res = []
        i, j = 0, 0
        m, n = len(left), len(right)
        while i < m and j < n:
            if left[i] < right[j]:
                res.append(left[i])
                i += 1
            else:
                res.append(right[j])
                j += 1
        res += left[i:] if i < m else right[j:]
        return res

    def sortArray(self, nums):
        if len(nums) <= 1:
            return nums
        mid = (len(nums)) // 2
        left = self.sortArray(nums[:mid])
        right = self.sortArray(nums[mid:])
        return self.mergeArray(left, right)

```



```

if __name__ == "__main__":
    nums = list(map(int, input().split()))
    solution = Solution()
    print(solution.sortArray(nums))

```

最长公共子序列

```

class Solution(object):
    def longestCommonSubsequence(self, text1, text2):
        m, n = len(text1), len(text2)
        # 空串情况自动初始化
        f = [[0] * (n+2) for _ in range(m+2)]
        for i in range(1, m+1):
            for j in range(1, n+1):
                if text1[i-1] == text2[j-1]:
                    f[i][j] = f[i-1][j-1] + 1
                else:
                    f[i][j] = max(f[i-1][j], f[i][j-1])
        return f[m][n]

if __name__ == "__main__":
    text1 = input()
    text2 = input()
    solution = Solution()
    print(solution.longestCommonSubsequence(text1, text2))

```

编辑距离

```

class Solution(object):
    def minDistance(self, word1, word2):
        m, n = len(word1), len(word2)
        f = [[0] * (n+2) for _ in range(m+2)]
        for i in range(1, m+1):
            f[i][0] += f[i-1][0] + 1
        for j in range(1, n+1):
            f[0][j] += f[0][j-1] + 1
        for i in range(1, m+1):
            for j in range(1, n+1):
                if word1[i-1] == word2[j-1]:
                    f[i][j] = f[i-1][j-1]
                else:
                    f[i][j] = min(f[i-1][j], f[i][j-1], f[i-1][j-1]) + 1
        return f[m][n]

if __name__ == "__main__":
    word1 = input()
    word2 = input()
    solution = Solution()
    print(solution.minDistance(word1, word2))

```

